Package ‘HapFinder’

December 12, 2025

Type Package

Title Microhaplotype Genotyping and Analysis Tool

Version 1.0.0

Description A comprehensive tool for microhaplotype genotyping,
forensic analysis, and interactive visualization through a
Shiny web interface.

License GPL-3

Encoding UTF-8

LazyData true

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.3

Depends R (>= 3.6.0),
shiny

Imports shinythemes,
shinycssloaders,
data.table,
dplyr,
stringr,
DT,
Rsamtools,
shinyjs,
shinyWidgets,
shinydisconnect,
shinydashboard,
shinyalert,
shinyFeedback

Contents

`analyzeMicrohaplotypeResults` *Analyze Microhaplotype Genotyping Results*

Description

This function analyzes microhaplotype genotyping output files to determine zygosity status, calculate allele coverage ratios (ACR), and provide comprehensive summary statistics for each locus.

Usage

```
analyzeMicrohaplotypeResults(file_path)
```

Arguments

`file_path`	Character string. Path to the microhaplotype calling output file. The file should be tab-separated with specific columns including "Tag", "Final_Microhaplotype_Genotype", and "Target_Microhaplotype_Genotype".

Value

A data frame with the following columns added to the original data:

- `Zygosity`: Zygosity status - "Homozygous", "Heterozygous", or "Low_depth_or_failed"
- `Haplotype_Depth_1`: Read depth for the first haplotype (if applicable)
- `Haplotype_Depth_2`: Read depth for the second haplotype (if heterozygous)
- `ACR`: Allele Coverage Ratio (larger depth / smaller depth) for heterozygous calls
- `ACR_Category`: Categorization of ACR value for quality assessment

Analysis Criteria

Homozygous When `Final_Microhaplotype_Genotype` contains exactly one haplotype

Heterozygous When `Final_Microhaplotype_Genotype` contains exactly two haplotypes

Low_depth_or_failed When `Final_Microhaplotype_Genotype` is "Low Reads", "Na", or contains no valid genotype

ACR Calculation

For heterozygous calls, the Allele Coverage Ratio (ACR) is calculated as:

$$ACR = \frac{\text{Larger haplotype depth}}{\text{Smaller haplotype depth}}$$

ACR values are categorized as:

- `Balanced (1.0-1.5)`: Near-equal coverage
- `Moderate imbalance (1.5-3.0)`: Moderate coverage difference
- `Severe imbalance (>3.0)`: Severe coverage difference
- `NA`: For homozygous or failed calls

Examples

```
## Not run:
# Analyze a microhaplotype calling output file
result <- analyzeMicrohaplotypeResults("Test_MHCalling.txt")

# View the summary statistics
print(result[, c("Tag", "Zygosity", "Haplotype_Depth_1", "Haplotype_Depth_2", "ACR")])

# Count different zygosity categories
table(result$Zygosity)

## End(Not run)
```

---

computeForensicStatsDetailed

Compute Detailed Forensic Statistics from Allele Frequencies

Description

Calculates comprehensive forensic genetic statistics from allele frequencies, including expected heterozygosity, polymorphism information content, effective number of alleles, discrimination power, probability of exclusion, and additional metrics like observed homozygosity and Shannon's information index. This function provides more detailed output than `computeForensicStats`.

Usage

```
computeForensicStatsDetailed(alle_freq)
```

Arguments

`alle_freq`	Numeric vector of allele frequencies. The vector should sum to 1 (within floating point tolerance) and all values should be between 0 and 1 inclusive.

Details

This function performs comprehensive forensic genetic analysis for a single genetic locus. It includes all calculations from `computeForensicStats` plus additional diversity metrics. The function validates input, normalizes frequencies if necessary, and provides detailed intermediate calculations.

The function computes the following statistics:

- **Observed Homozygosity**: Probability that two randomly selected alleles are identical, calculated as $\sum p_i^2$.
- **Expected Heterozygosity (He)**: Also known as gene diversity, calculated as $1 - \sum p_i^2$ where p_i is the frequency of the i-th allele.
- **Polymorphism Information Content (PIC)**: Measures the informativeness of a marker, calculated as $1 - \sum p_i^2 - (\sum p_i^2)^2 + \sum p_i^4$.
- **Effective Number of Alleles (Ae)**: Reciprocal of homozygosity, calculated as $1/\sum p_i^2$. Represents the number of equally frequent alleles that would give the same homozygosity.
- **Discrimination Power (DP)**: Probability that two randomly selected individuals have different genotypes, calculated as $1 - \sum_{i=1}^{n} \sum_{j=1}^{n} (p_i p_j)^2 k_{ij}$ where $k_{ij} = 1$ if $i = j$, else $k_{ij} = 4$.
- **Probability of Exclusion (PE)**: Probability that a random unrelated individual would be excluded as the parent/contributor, calculated as $\sum_i p_i(1 - p_i)^2 - 0.5 \cdot \sum_{i \neq j} (p_i p_j)^2 (4 - 3p_i - 3p_j)$.
- **Shannon's Information Index**: A diversity measure from information theory, calculated as $-\sum p_i \log(p_i)$.

Value

A list containing the following elements:

allele_frequencies The input allele frequencies (after normalization)

number_of_alleles Total number of alleles in the locus

homozygosity Observed homozygosity ($\sum p_i^2$)

heterozygosity Expected heterozygosity (He)

polymorphism_information_content Polymorphism information content (PIC)

effective_number_of_alleles Effective number of alleles (Ae)

discrimination_power Discrimination power (DP)

probability_of_exclusion Probability of exclusion (PE)

shannon_index Shannon's information index (diversity measure)

summary_string A text summary of the main statistics

all_statistics Named vector containing He, PIC, Ae, DP, and PE (rounded to 6 decimal places)

intermediate_calculations List containing intermediate values used in calculations

Input Requirements

- Allele frequencies must be provided as a numeric vector
- Frequencies should sum to 1 (within `1e-10` tolerance)
- All frequencies must be between 0 and 1 inclusive
- Missing values (NA) are removed with warning

Calculation Details

The function performs the following steps:

1. Input validation and frequency normalization
2. Calculation of basic summary statistics (number of alleles, homozygosity)
3. Computation of five core forensic statistics (He, PIC, Ae, DP, PE)
4. Calculation of additional diversity metrics (Shannon's index)
5. Compilation of results in structured format

All calculations use double precision floating-point arithmetic. Results are rounded to 6 decimal places for display, but full precision is maintained in intermediate calculations.

Mathematical Formulas

For a locus with alleles $i = 1, 2, ..., n$ and frequencies p_i:

$$\text{Homozygosity} = \sum_{i=1}^{n} p_i^2$$

$$He = 1 - \sum_{i=1}^{n} p_i^2$$

$$PIC = 1 - \sum_{i=1}^{n} p_i^2 - \left(\sum_{i=1}^{n} p_i^2\right)^2 + \sum_{i=1}^{n} p_i^4$$

$$Ae = \frac{1}{\sum_{i=1}^{n} p_i^2}$$

$$DP = 1 - \sum_{i=1}^{n} \sum_{j=1}^{n} (p_i p_j)^2 \cdot k_{ij}$$

where $k_{ij} = 1$ if $i = j$, else $k_{ij} = 4$

$$PE = \sum_{i} p_i(1 - p_i)^2 - 0.5 \cdot \sum_{i \neq j} (p_i p_j)^2 (4 - 3p_i - 3p_j)$$

$$\text{Shannon's Index} = -\sum_{i=1}^{n} p_i \log(p_i)$$

See Also

`computeForensicStats` for a simpler output format, `.validateAlleleFrequencies` for input validation.

Examples

```
# Example 1: Simple biallelic marker
computeForensicStatsDetailed(c(0.7, 0.3))

# Example 2: Tetranucleotide marker with four alleles
computeForensicStatsDetailed(c(0.4, 0.3, 0.2, 0.1))

# Example 3: Highly polymorphic marker
computeForensicStatsDetailed(c(0.25, 0.20, 0.15, 0.15, 0.10, 0.10, 0.05))

# Example 4: Using allele counts to compute frequencies
allele_counts <- c(50, 30, 20)  # Observed counts for three alleles
frequencies <- allele_counts / sum(allele_counts)
computeForensicStatsDetailed(frequencies)

# Example 5: Accessing specific results
results <- computeForensicStatsDetailed(c(0.6, 0.3, 0.1))
results$heterozygosity  # Expected heterozygosity
results$discrimination_power  # Discrimination power
results$summary_string  # Text summary
```

MicrohapCalling *Microhaplotype Calling from BAM Files*

Description

This function performs comprehensive microhaplotype calling from BAM files for targeted sequencing data. It processes both paired-end and single-end sequencing data to genotype microhaplotype loci using alignment information, quality scores, and variant detection from CIGAR strings and MD tags.

Usage

```
MicrohapCalling(
  MH_info,
  bam.file,
  wd_link,
  lian_qual,
  base_qual,
  base_qual_include,
  other_snp_threshold,
  MH_read_threshold,
  MH_ratio_threshold,
  SNP_ratio_threshold,
  PE_data
)
```

Arguments

`MH_info`	Path to the microhaplotype information file (tab-separated). The file should contain columns: Chr, Start, End, VC (variant positions), VC_type (variant types), Tar_end, Tar_sat, Tag.
`bam.file`	BAM file name (without full path).
`wd_link`	Working directory path where results will be stored.
`lian_qual`	Read quality threshold for mean Phred quality score.
`base_qual`	Minimum base quality threshold for variant calling.
`base_qual_include`	Whether to include base quality filtering ("T" for TRUE, "F" for FALSE).
`other_snp_threshold`	Frequency threshold for reporting non-target SNPs in the region.
`MH_read_threshold`	Minimum read count threshold for reporting a microhaplotype.
`MH_ratio_threshold`	Frequency threshold for microhaplotype calling.
`SNP_ratio_threshold`	Frequency threshold for SNP calling.
`PE_data`	Whether the data is paired-end ("T") or single-end ("F").

Details

During execution, the function creates two subdirectories within `wd_link`:

- `Result/`: Contains consolidated microhaplotype calling results. The main result file is named with the pattern: `Processed_bam_<timestamp>_<randomID>_MHCalling.txt`
- `Sample/`: Contains detailed per-locus and per-read analysis results (format: Tag_all_Result_PairHMM.txt). For each microhaplotype locus, a subdirectory is created with the locus tag, containing raw read information, variant calling details, intermediate genotype calls, and quality control metrics for each read (both paired-end and single-end data).

The function performs the following main steps:

1. Loads microhaplotype information from the specified file.
2. Sets up working directories for results and intermediate files.
3. For each microhaplotype locus:
    - Extracts genomic region information.
    - Filters BAM file to the target region.
    - Parses CIGAR strings to determine read alignment.
    - Parses MD tags to identify variants.
    - Calls genotypes for target variant positions.
    - Applies quality filters at read and base levels.
    - For paired-end data, merges and compares forward and reverse reads.
    - Summarizes microhaplotype and SNP-level results.

    - Applies frequency thresholds for final calling.
4. Writes comprehensive results to output files.

The function handles both paired-end and single-end data with appropriate processing for each. For paired-end data, it performs additional consistency checks between forward and reverse reads and merges results when concordant.

Value

The path to the result file containing microhaplotype calling results. The result file is a tab-separated table with columns:

- `Tag`: Microhaplotype locus identifier
- `Chr`: Chromosome
- `Reads`: Total number of reads covering the locus
- `Target_Microhaplotype_Genotype`: Genotype for target microhaplotype positions
- `Target_SNP_Genotype`: SNP-level genotypes with frequencies
- `Other_SNPs_in_Target_Region`: Non-target SNPs detected in the region with frequencies
- `Haplotypes_in_Target_Region`: All haplotypes observed in the region with read counts
- `Final_Microhaplotype_Genotype`: Final called microhaplotype genotype

File Formats

Microhaplotype Information File: Tab-separated file with the following required columns:

- `Chr`: Chromosome name (e.g., "chr1")
- `Start`: Start position of the microhaplotype region
- `End`: End position of the microhaplotype region
- `VC`: Variant positions, comma-separated (e.g., "116204807,116204844")
- `VC_type`: **Reference genome bases** at each variant position, comma-separated. For example, if VC is "116204807,116204844" and the reference has "A" at 116204807 and "G" at 116204844, then VC_type should be "A,G".
- `Tar_end`: Target end position for analysis
- `Tar_sat`: Target satellite information
- `Tag`: Unique locus identifier (e.g., "MH-1")

Output Result File: Tab-separated file with comprehensive calling results including read counts, genotypes, frequencies, and quality metrics.

Examples

```
## Not run:
# Example for paired-end data
result_file <- shinyappMHCalling(
  MH_info = "microhaplotypes_info.txt",
  bam.file = "sample1.bam",
```

```
  wd_link = "/path/to/results",
  lian_qual = 20,
  base_qual = 30,
  base_qual_include = "F",
  other_snp_threshold = 0.01,
  MH_read_threshold = 10,
  MH_ratio_threshold = 0.25,
  SNP_ratio_threshold = 0.25,
  PE_data = "T"
)

# Read and inspect results
results <- read.table(result_file, header = TRUE, sep = "\t")
head(results)

## End(Not run)
```

`MixMicrohapCall` *Microhaplotype Calling from BAM Files for Mixture Sequencing Data*

Description

This function performs comprehensive microhaplotype calling from BAM files for mixture sequencing data. It processes both paired-end and single-end sequencing data to genotype microhaplotype loci using alignment information, quality scores, and variant detection from CIGAR strings and MD tags.

Usage

```
MixMicrohapCall(
  MH_info,
  bam.file,
  wd_link,
  lian_qual,
  base_qual,
  base_qual_include,
  MH_read_threshold,
  PE_data
)
```

Arguments

`MH_info`	Path to the microhaplotype information file (tab-separated). The file should contain columns: Chr, Start, End, VC (variant positions), VC_type (variant types), Tar_end, Tar_sat, Tag.
`bam.file`	BAM file name or path to the BAM file.
`wd_link`	Working directory path where results will be stored.

lian_qual — Read quality threshold for mean Phred quality score.

base_qual — Minimum base quality threshold for variant calling.

base_qual_include — Whether to include base quality filtering ("T" for TRUE, "F" for FALSE).

MH_read_threshold — Minimum read count threshold for reporting a microhaplotype.

PE_data — Whether the data is paired-end ("T") or single-end ("F").

Details

This function implements a comprehensive pipeline for microhaplotype analysis in mixture sequencing data. It handles both paired-end and single-end sequencing data with appropriate quality filtering and variant calling mechanisms.

During execution, the function creates two subdirectories within `wd_link`:

- `Result/`: Contains consolidated microhaplotype calling results. The main result file is named with the pattern: `Processed_bam_<timestamp>_<randomID>_MixMHCall.txt`
- `Sample/`: Contains detailed per-locus analysis results (format: Tag_all_Result_PairHMM.txt). For each microhaplotype locus, a subdirectory is created with the locus tag, containing raw read information, variant calling details, intermediate genotype calls, and quality control metrics for each read.

The function performs the following main steps:

1. Loads microhaplotype information from the specified file.
2. Sets up working directories for results and intermediate files.
3. For each microhaplotype locus:
   - Extracts genomic region information.
   - Filters BAM file to the target region.
   - Parses CIGAR strings to determine read alignment.
   - Parses MD tags to identify variants.
   - Calls genotypes for target variant positions.
   - Applies quality filters at read and base levels.
   - For paired-end data, merges and compares forward and reverse reads.
   - Summarizes microhaplotype results.
   - Applies read count threshold for reporting.
4. Returns comprehensive results as a data frame.

For paired-end data, the function performs additional consistency checks between forward and reverse reads and merges results when concordant. For single-end data, it processes each read independently.

Value

A data frame containing microhaplotype calling results with columns:

- Tag: Microhaplotype locus identifier
- Chr: Chromosome
- Reads: Total number of reads covering the locus
- Target_Microhaplotype_Genotype: Genotype for target microhaplotype positions
- Haplotypes_in_Target_Region: All haplotypes observed in the region with read counts

File Formats

Microhaplotype Information File: Tab-separated file with the following required columns:

- Chr: Chromosome name (e.g., "chr1")
- Start: Start position of the microhaplotype region
- End: End position of the microhaplotype region
- VC: Variant positions, hyphen-separated (e.g., "116204807-116204844")
- VC_type: **Reference genome bases** at each variant position, hyphen-separated. For example, if VC is "116204807-116204844" and the reference has "A" at 116204807 and "G" at 116204844, then VC_type should be "A-G".
- Tar_end: Target end position for analysis
- Tar_sat: Target satellite information
- Tag: Unique locus identifier (e.g., "MH-1")

Examples

```
## Not run:
# Example for paired-end mixture data
result <- MixMicrohapCall(
  MH_info = "microhaplotypes_info.txt",
  bam.file = "mixture_sample.bam",
  wd_link = "/path/to/results",
  lian_qual = 20,
  base_qual = 30,
  base_qual_include = "F",
  MH_read_threshold = 10,
  PE_data = "T"
)

# Inspect results
head(result)

# Example for single-end mixture data
result_se <- MixMicrohapCall(
  MH_info = "microhaplotypes_info.txt",
  bam.file = "mixture_sample_se.bam",
  wd_link = "/path/to/results",
```

```
  lian_qual = 20,
  base_qual = 30,
  base_qual_include = "T",
  MH_read_threshold = 5,
  PE_data = "F"
)

## End(Not run)
```

---

popMicroHapFreq *Calculate Population Frequencies for Microhaplotype Loci*

Description

This function processes individual microhaplotype genotype files to calculate population-level haplotype frequencies across multiple loci. It is designed to work with output files from microhaplotype calling software, converting individual genotype data into population frequency estimates.

Usage

```
popMicroHapFreq(input_link, MH_info, ind_file, out_link)
```

Arguments

`input_link` Character string specifying the directory path containing input individual genotype files generated by microhaplotype calling software. The input files must be tab-delimited text files where each row represents a microhaplotype locus. The genotype data for each individual should not contain NA values or more than two haplotypes per locus.

`MH_info` Character string specifying the file path to the microhaplotype information file. This file should contain at least a 'Tag' column with locus identifiers that match those in the individual genotype files.

`ind_file` Character string specifying the file path to the individual information file. This file should contain at least two columns:

- `file_name`: The name of the corresponding individual genotype file in the `input_link` directory
- `Ind_name`: The unique identifier for each individual. These values will be used as column headers in the output individual haplotype matrix.

The order of rows in this file determines the order of columns in the output matrix.

`out_link` Character string specifying the output directory path where result files will be saved. The directory will be created if it doesn't exist.

Value

A data frame containing population frequencies for all microhaplotype loci with the following columns:

- `mh_locus`: Microhaplotype locus identifier
- `haplotype`: Observed haplotype sequence
- `count`: Number of times the haplotype was observed
- `frequency`: Population frequency of the haplotype

Additionally, two files are written to the output directory:

- `"Ind_all_MH_haplotypes.txt"`: A matrix where rows represent microhaplotype loci (named by Tag), columns represent individuals (named by Ind_name), and each cell contains the individual's genotype for that locus. Genotypes are represented as two haplotypes separated by a newline character.
- `"MH_haplotypes_popFreq.txt"`: The detailed frequency table containing the same information as the returned data frame.

Note

The function expects genotype files where missing data (NA values) and genotypes with more than two haplotypes have been filtered out prior to analysis.

See Also

`toEuroForMixFreqData` for converting the output to EuroForMix Frequency database format.

Examples

```
## Not run:
# Example usage with typical file paths
popMicroHapFreq(
  input_link = "./input_genotypes/",
  MH_info = "./reference/MH_info.txt",
  ind_file = "./samples/individuals.txt",
  out_link = "./output/"
)

## End(Not run)
```

recalculateMHcallsByThreshold

Recalculate Microhaplotype Calls Based on Adjusted Thresholds

Description

This function recalculates microhaplotype (MH) calling results by applying user-defined read count and frequency thresholds to raw haplotype data. It processes paired-end and single-end sequencing data differently and generates updated MH calling results.

Usage

```
recalculateMHcallsByThreshold(
  MHcallout,
  callpath,
  MH_info,
  out_path,
  MH_read_threshold,
  MH_ratio_threshold,
  PE_data
)
```

Arguments

- `MHcallout` Character string. Path to the existing MH calling output file generated by the primary analysis pipeline.
- `callpath` Character string. Directory path containing intermediate result files (in /path/to/Sample/) for each microhaplotype locus (format: Tag_all_Result.txt).
- `MH_info` Character string. Path to the microhaplotype information file (tab-separated) containing locus definitions and annotations.
- `out_path` Character string. Directory path where output files will be saved.
- `MH_read_threshold` Numeric. Minimum read count required for a haplotype to be considered in the analysis.
- `MH_ratio_threshold` Numeric. Minimum frequency threshold (0-1) for haplotype calling. Haplotypes below this proportion will be filtered out.
- `PE_data` Character string. Indicates whether data is paired-end ("T") or single-end ("F").

Value

The function does not return a value directly but modifies the input MHcallout data frame in-place and writes the updated results to a file. Output files are saved in the specified out_path directory.

Input File Requirements

- `MHcallout`: Tab-separated file with columns including "Target_Microhaplotype_Genotype", "Haplotypes_in_Target_Region", and "Final_Microhaplotype_Genotype".
- `MH_info`: Tab-separated file with at least a "Tag" column identifying microhaplotype loci.
- Intermediate files: Each locus should have a corresponding file named Tag_all_Result.txt in the callpath directory.

Processing Steps

1. Loads microhaplotype information and existing calling results.
2. For each locus, loads the corresponding intermediate result file.
3. Filters haplotypes based on read count threshold.
4. Calculates haplotype frequencies and applies frequency threshold.
5. Updates calling results with filtered haplotypes.
6. Writes progress messages during processing.

Examples

```
## Not run:
# Example usage for paired-end data
recalculateMHcallsByThreshold(
  MHcallout = "/path/to/Text_MHCalling.txt",
  callpath = "/path/to/Sample/",
  MH_info = "/path/to/microhaplotype_info.txt",
  out_path = "/path/to/output/",
  MH_read_threshold = 10,
  MH_ratio_threshold = 0.3,
  PE_data = "T"
)

# Example usage for single-end data
recalculateMHcallsByThreshold(
  MHcallout = "/path/to/previous_results.txt",
  callpath = "/path/to/Sample/",
  MH_info = "/path/to/microhaplotype_info.txt",
  out_path = "/path/to/output/",
  MH_read_threshold = 5,
  MH_ratio_threshold = 0.25,
  PE_data = "F"
)

## End(Not run)
```

`runHapFinder` *Launch the HapFinder Shiny Application*

Description

This function launches the interactive Shiny application for microhaplotype analysis. The application provides a user-friendly interface for data upload, parameter setting, analysis execution, and result visualization.

Usage

```
runHapFinder(launch.browser = TRUE, port = 3838, host = "127.0.0.1")
```

Arguments

`launch.browser`	Logical. If TRUE, launch in browser. Default is TRUE.
`port`	Integer. Port number to launch the app. Default is 3838.
`host`	Character. Host address. Default is "127.0.0.1".

Value

This function does not return a value; it launches a Shiny app.

Examples

```
## Not run:
# Launch the app in default browser
HapFinder::runHapFinder()

# Launch on specific port
HapFinder::runHapFinder(port = 4242)

# Launch without opening browser
HapFinder::runHapFinder(launch.browser = FALSE)

## End(Not run)
```

`shinyappMHCalling` *Microhaplotype Calling from BAM Files*

Description

This function performs comprehensive microhaplotype calling from BAM files for targeted sequencing data via shiny app. It processes both paired-end and single-end sequencing data to genotype microhaplotype loci using alignment information, quality scores, and variant detection from CIGAR strings and MD tags.

Usage

```
shinyappMHCalling(
  MH_info,
  bam.file,
  wd_link,
  lian_qual,
  base_qual,
  base_qual_include,
  other_snp_threshold,
  MH_read_threshold,
  MH_ratio_threshold,
  SNP_ratio_threshold,
  PE_data
)
```

Arguments

`MH_info`	Path to the microhaplotype information file (tab-separated). The file should contain columns: Chr, Start, End, VC (variant positions), VC_type (variant types), Tar_end, Tar_sat, Tag.
`bam.file`	BAM file name (without full path).
`wd_link`	Working directory path where results will be stored.
`lian_qual`	Read quality threshold for mean Phred quality score.
`base_qual`	Minimum base quality threshold for variant calling.
`base_qual_include`	Whether to include base quality filtering ("T" for TRUE, "F" for FALSE).
`other_snp_threshold`	Frequency threshold for reporting non-target SNPs in the region.
`MH_read_threshold`	Minimum read count threshold for reporting a microhaplotype.
`MH_ratio_threshold`	Frequency threshold for microhaplotype calling.
`SNP_ratio_threshold`	Frequency threshold for SNP calling.
`PE_data`	Whether the data is paired-end ("T") or single-end ("F").

Details

During execution, the function creates two subdirectories within `wd_link`:

- `Result/`: Contains consolidated microhaplotype calling results. The main result file is named with the pattern: `Processed_bam_<timestamp>_<randomID>_MHCalling.txt`
- `Sample/`: Contains detailed per-locus and per-read analysis results (format: Tag_all_Result_PairHMM.txt). For each microhaplotype locus, a subdirectory is created with the locus tag, containing raw read information, variant calling details, intermediate genotype calls, and quality control metrics for each read (both paired-end and single-end data).

The function performs the following main steps:

1. Loads microhaplotype information from the specified file.
2. Sets up working directories for results and intermediate files.
3. For each microhaplotype locus:
   - Extracts genomic region information.
   - Filters BAM file to the target region.
   - Parses CIGAR strings to determine read alignment.
   - Parses MD tags to identify variants.
   - Calls genotypes for target variant positions.
   - Applies quality filters at read and base levels.
   - For paired-end data, merges and compares forward and reverse reads.
   - Summarizes microhaplotype and SNP-level results.
   - Applies frequency thresholds for final calling.
4. Writes comprehensive results to output files.

The function handles both paired-end and single-end data with appropriate processing for each. For paired-end data, it performs additional consistency checks between forward and reverse reads and merges results when concordant.

Value

The path to the result file containing microhaplotype calling results. The result file is a tab-separated table with columns:

- `Tag`: Microhaplotype locus identifier
- `Chr`: Chromosome
- `Reads`: Total number of reads covering the locus
- `Target_Microhaplotype_Genotype`: Genotype for target microhaplotype positions
- `Target_SNP_Genotype`: SNP-level genotypes with frequencies
- `Other_SNPs_in_Target_Region`: Non-target SNPs detected in the region with frequencies
- `Haplotypes_in_Target_Region`: All haplotypes observed in the region with read counts
- `Final_Microhaplotype_Genotype`: Final called microhaplotype genotype

File Formats

Microhaplotype Information File: Tab-separated file with the following required columns:

- `Chr`: Chromosome name (e.g., "chr1")
- `Start`: Start position of the microhaplotype region
- `End`: End position of the microhaplotype region
- `VC`: Variant positions, comma-separated (e.g., "116204807,116204844")
- `VC_type`: **Reference genome bases** at each variant position, comma-separated. For example, if VC is "116204807,116204844" and the reference has "A" at 116204807 and "G" at 116204844, then VC_type should be "A,G".

- Tar_end: Target end position for analysis
- Tar_sat: Target satellite information
- Tag: Unique locus identifier (e.g., "MH-1")

Output Result File: Tab-separated file with comprehensive calling results including read counts, genotypes, frequencies, and quality metrics.

Examples

```
## Not run:
# Example for paired-end data
result_file <- shinyappMHCalling(
  MH_info = "microhaplotypes_info.txt",
  bam.file = "sample1.bam",
  wd_link = "/path/to/results",
  lian_qual = 20,
  base_qual = 30,
  base_qual_include = "F",
  other_snp_threshold = 0.01,
  MH_read_threshold = 10,
  MH_ratio_threshold = 0.25,
  SNP_ratio_threshold = 0.25,
  PE_data = "T"
)

# Read and inspect results
results <- read.table(result_file, header = TRUE, sep = "\t")
head(results)

## End(Not run)
```

---

toEuroForMixFreqData *Convert Population Frequency Data to EuroForMix Format*

Description

This function transforms microhaplotype population frequency data into a format compatible with EuroForMix database structure. The output format is a matrix with alleles as rows and loci as columns.

Usage

```
toEuroForMixFreqData(popfreq, out_link)
```

Arguments

popfreq	Character string specifying the file path to the population frequency file generated by popMicroHapFreq.
out_link	Character string specifying the output directory path.

Value

A matrix in EuroForMix format with alleles as rows, loci as columns, and frequencies as cell values. The first column contains allele names. The matrix is also written to disk as "EuroFreqdatabase.txt".

Examples

```
## Not run:
toEuroForMixFreqData(
  popfreq = "./output/MH_haplotypes_popFreq.txt",
  out_link = "./euroformat/"
)

## End(Not run)
```

---

toEuroforMixInput — *Convert Mixture Microhaplotype Calling Results to EuroForMix Input Format*

Description

This function processes mixture microhaplotype genotyping results and converts them into the input format required by EuroForMix, a forensic DNA mixture analysis software.

Usage

```
toEuroforMixInput(result_mix, mix_name, Mix_read_threshold, wd_link)
```

Arguments

`result_mix` A data frame containing results by MixMicrohapCall function. The data frame must include at least two columns: "Tag" (microhaplotype locus identifier) and "Target_Microhaplotype_Genotype" (genotype information with read counts).

`mix_name` Character string specifying the name of the mixture sample. This name will be used in the output file and as the SampleName in the EuroForMix input file.

`Mix_read_threshold` Integer specifying the minimum read count threshold for including an allele in the analysis. Alleles with read counts below this threshold will be excluded.

`wd_link` Character string specifying the working directory path where the output file will be saved.

Details

This function performs the following steps:

1. Parses genotype information from the Target_Microhaplotype_Genotype column
2. Filters out alleles containing 'X' or 'R' (indicating ambiguous calls)

3. Applies the read count threshold to exclude low-coverage alleles
4. Formats the data into the specific structure required by EuroForMix
5. Writes the formatted data to a text file

The output format follows EuroForMix requirements with columns:

- SampleName: The mixture sample identifier
- Marker: Microhaplotype locus identifier
- Allele1, Allele2, ...: Allele sequences
- Height1, Height2, ...: Corresponding read counts (peak heights)

Value

A data frame in EuroForMix input format. The function also writes a tab-separated text file to the specified directory with the name pattern: `<mix_name>_Euro_mixture_input.txt`.

Examples

```
## Not run:
# Load microhaplotype calling results
result_mix <- read.table("Test_MHCalling.txt",
                         header = TRUE, sep = "\t")

# Convert to EuroForMix format
euroforMix_input <- toEuroforMix(
  result_mix = result_mix,
  mix_name = "Case001",
  Mix_read_threshold = 10,
  wd_link = "/path/to/output/directory/"
)

# The function creates a file named "Case001_Euro_mixture_input.txt"
# and returns the data frame for further use

## End(Not run)
```

`toIndReference` *Convert Individual Microhaplotype Genotype Files to Individual Reference Format*

Description

This function processes individual microhaplotype genotype files and converts them into a standardized reference format with four columns: SampleName, Marker, Allele1, and Allele2. The output format is specifically designed for use with EuroForMix and other forensic mixture analysis software. Each individual's genotype data is saved as a separate file in the output directory.

Usage

```
toIndReference(input_link, MH_info, ind_file, out_link)
```

Arguments

- `input_link`: Character string specifying the directory path containing input individual genotype files generated by microhaplotype calling software. The input files must be tab-delimited text files where each row represents a microhaplotype locus. The genotype data for each individual should not contain NA values or more than two haplotypes per locus.
- `MH_info`: Character string specifying the file path to the microhaplotype information file. This file should contain at least a 'Tag' column with locus identifiers that match those in the individual genotype files.
- `ind_file`: Character string specifying the file path to the individual information file. This file should contain at least two columns:
  - `file_name`: The name of the corresponding individual genotype file in the `input_link` directory
  - `Ind_name`: The unique identifier for each individual
- `out_link`: Character string specifying the output directory path where individual reference files will be saved.

Value

This function does not return a value but writes individual reference files to the specified output directory. Each file is named using the individual's identifier (e.g., "ind1.txt") and contains the following columns:

- `SampleName`: Individual identifier
- `Marker`: Microhaplotype locus identifier (Tag)
- `Allele1`: First allele/haplotype
- `Allele2`: Second allele/haplotype (may be empty for homozygotes)

Note

The function processes each individual file separately and performs the following steps:

1. Reads the individual genotype file
2. Filters rows to include only loci specified in MH_info
3. Parses genotype strings to extract alleles
4. Cleans allele strings by removing numbers and punctuation
5. Formats data into the four-column reference format
6. Writes output file for each individual

Empty allele strings may occur when genotypes are homozygous or when data cleaning results in empty values.

See Also

popMicroHapFreq for calculating population frequencies

Examples

```
## Not run:
# Example usage
toIndReference(
  input_link = "./input_genotypes/",
  MH_info = "./reference/MH_info.txt",
  ind_file = "./samples/individuals.txt",
  out_link = "./output/individual_references/"
)

## End(Not run)
```

Index